

Implementation Report: E-Voting Application

Tumusiime Julius
Lwanyaga Ivan
Mucunguzi Godfrey
John Paul
Uganda Christian University

March 2026

1 Introduction

This report explains the implementation of the **E-Voting Application** and how the system was designed according to the required development criteria. The application is a console-based Python system that allows administrators to manage elections and voters to participate in the voting process.

The system supports two types of users:

- Administrators – responsible for managing elections, candidates, and voters.
- Voters – responsible for registering, logging in, and casting votes.

The application uses **JSON files for data persistence** and includes security mechanisms such as **password hashing and audit logging** to protect user data and ensure transparency in the voting process.

2 Implementation Based on Development Criteria

2.1 Modular Structure

The application follows a modular structure where the system is divided into separate files that each perform a specific role. This improves the organization and maintainability of the application.

Main Modules

- **main.py**
Acts as the entry point of the application. It controls the program flow, user dashboards, and system interaction.

- **models.py**

Defines the data models used in the system, including classes such as `User`, `Admin`, `Voter`, `Candidate`, `Poll`, and `Vote`. It also contains the `DataManager` class responsible for data storage and retrieval.

- **services.py**

Contains the business logic of the system, including authentication and candidate management operations.

- **ui.py**

Handles the user interface components such as menus, prompts, and formatted console output.

- **evoting_data.json**

Stores application data so that it persists between program executions.

Each module performs a single responsibility, which helps maintain a clear structure and allows easier modification or extension of the system.

2.2 Object-Oriented Design

The application uses object-oriented programming principles to create a structured and reusable design.

The system includes the following classes:

- `User`
- `Admin`
- `Voter`
- `Candidate`
- `Poll`
- `Vote`
- `EVotingApp`

Inheritance

The `Admin` and `Voter` classes inherit common properties and behaviors from the base `User` class.

Encapsulation

Attributes and related methods are contained within classes, ensuring that internal data is protected and accessed through defined methods.

Meaningful Methods

Classes include methods that perform specific operations, such as converting objects to dictionary format for JSON storage.

Using object-oriented design helps organize the system into reusable and maintainable components.

2.3 Separation of Concerns

The system separates different responsibilities into distinct layers to improve maintainability.

Data Layer

The data layer is responsible for storing and retrieving information. This functionality is implemented in the `DataManager` class within `models.py`.

Logic Layer

The logic layer contains the core system operations such as authentication and election management. These functions are implemented in `services.py`.

User Interface Layer

The user interface layer manages interaction with the user through menus and prompts. This is implemented in `ui.py`.

This separation ensures that the user interface does not directly manipulate stored data and that the logic layer does not control how information is displayed.

2.4 Clean Code Quality

The application follows good coding practices to improve readability and maintainability.

Clear Naming Conventions

Variables and functions follow Python's `snake_case` naming convention and use descriptive names.

Readable Structure

Functions are kept short and focused on performing a single task.

No Code Duplication

Reusable utility functions are used throughout the application to avoid repeated code.

Error Handling

Input validation and exception handling are implemented to prevent system crashes and ensure reliable operation.

Comments and Documentation

Comments and docstrings are used to explain important parts of the code and improve understanding for other developers.

2.5 Working Application

The application is fully functional and allows both administrators and voters to perform their required tasks.

Administrator Capabilities

- Create, update, and delete candidates
- Manage polling stations
- Create and manage election positions
- Assign candidates to polls
- Open and close polls
- Manage voter and administrator accounts

Voter Capabilities

- Register accounts
- Log in to the system
- Vote in active polls
- View voting history
- View and update profile information

Security Features

- Password hashing
- Voter verification
- Audit logging
- Account deactivation

All data is stored in a JSON file so that it can be loaded whenever the application starts again.

2.6 Explanation of Structure and Design Decisions

Several design decisions were made during development.

JSON Storage

JSON was chosen because it is lightweight and easy to implement for a small application.

Console Interface

A console-based interface simplifies development and allows focus on implementing core system functionality.

Service-Based Architecture

Separating business logic into service classes makes it easier to extend the system in the future.

Audit Logging

Audit logs help track important actions in the system, improving transparency and accountability.

3 Conclusion

The E-Voting application demonstrates a well-structured implementation using modular programming, object-oriented design, and clear separation of concerns.

The system successfully supports key election management features including user management, poll management, and secure voting.

Future improvements could include adding automated testing, developing a graphical interface, and migrating data storage from JSON to a relational database for larger-scale use.